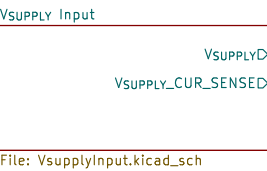
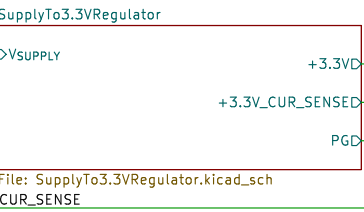


HackerDAQ Main Unit

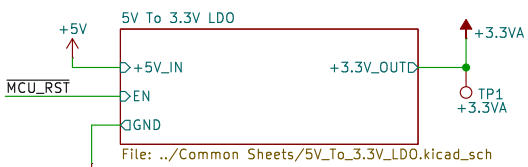
Power Input



Main 3.3V Supply

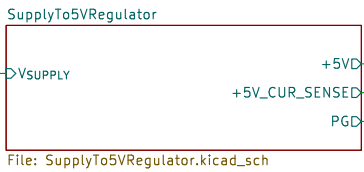


3.3V Analog Supply

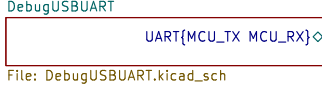


MCX N54 datasheet says that VDD_P4/VDD_ANA cannot turn on before the main power. So, enabling this regulator off the power good line from +3.3V, which is jumpered with MCU_RST

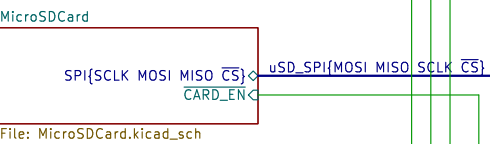
Main 5V Supply



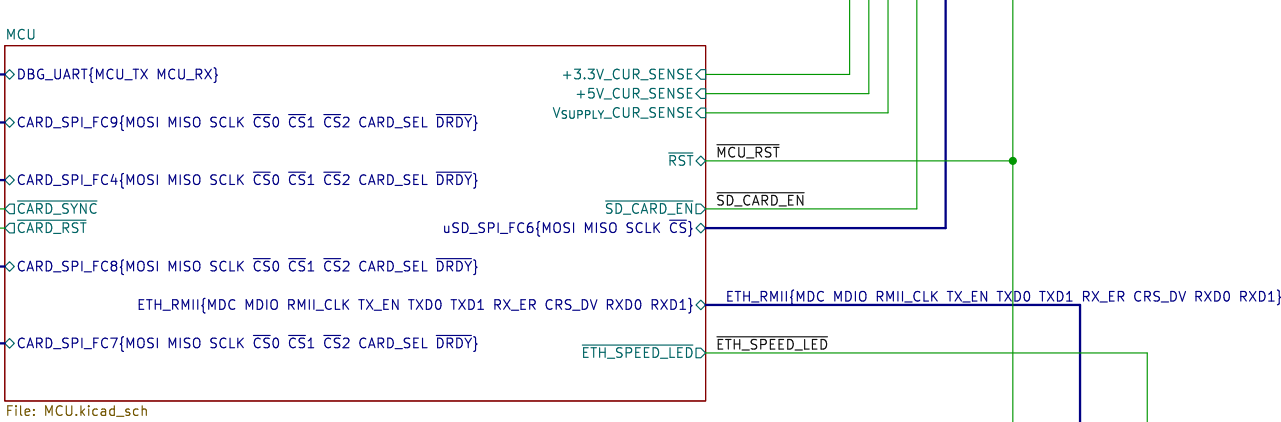
USB to UART for Debugging and Flashing



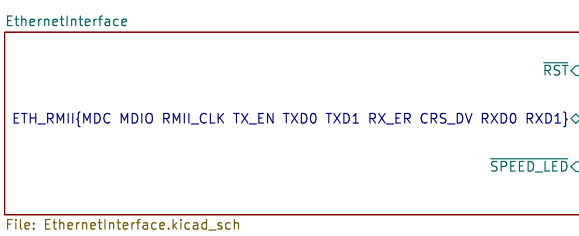
SD Card



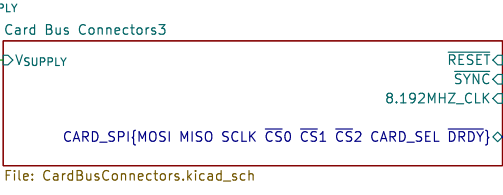
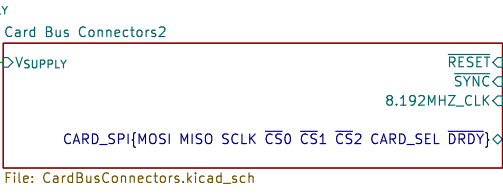
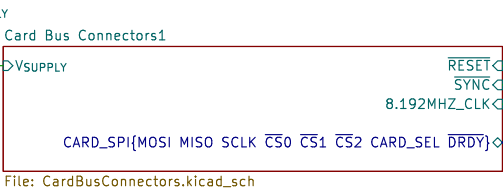
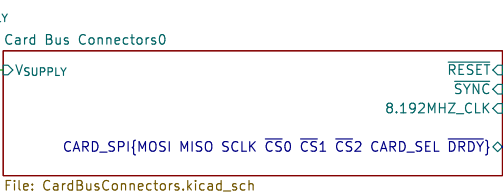
MCU



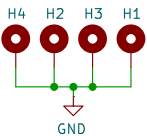
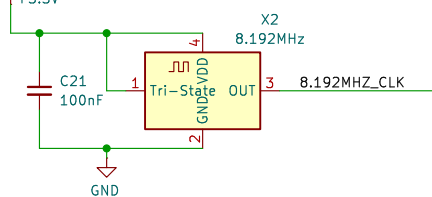
Ethernet

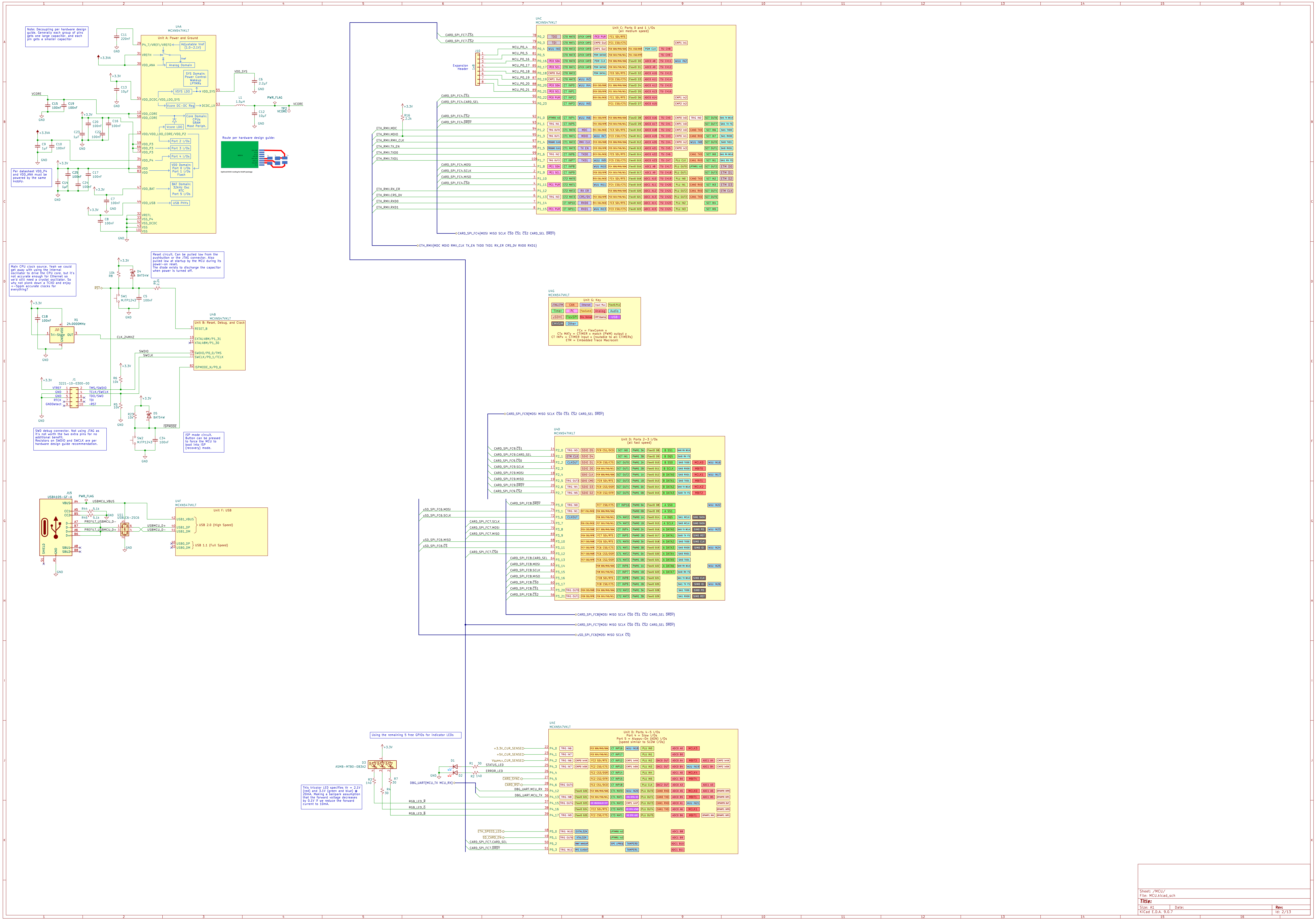


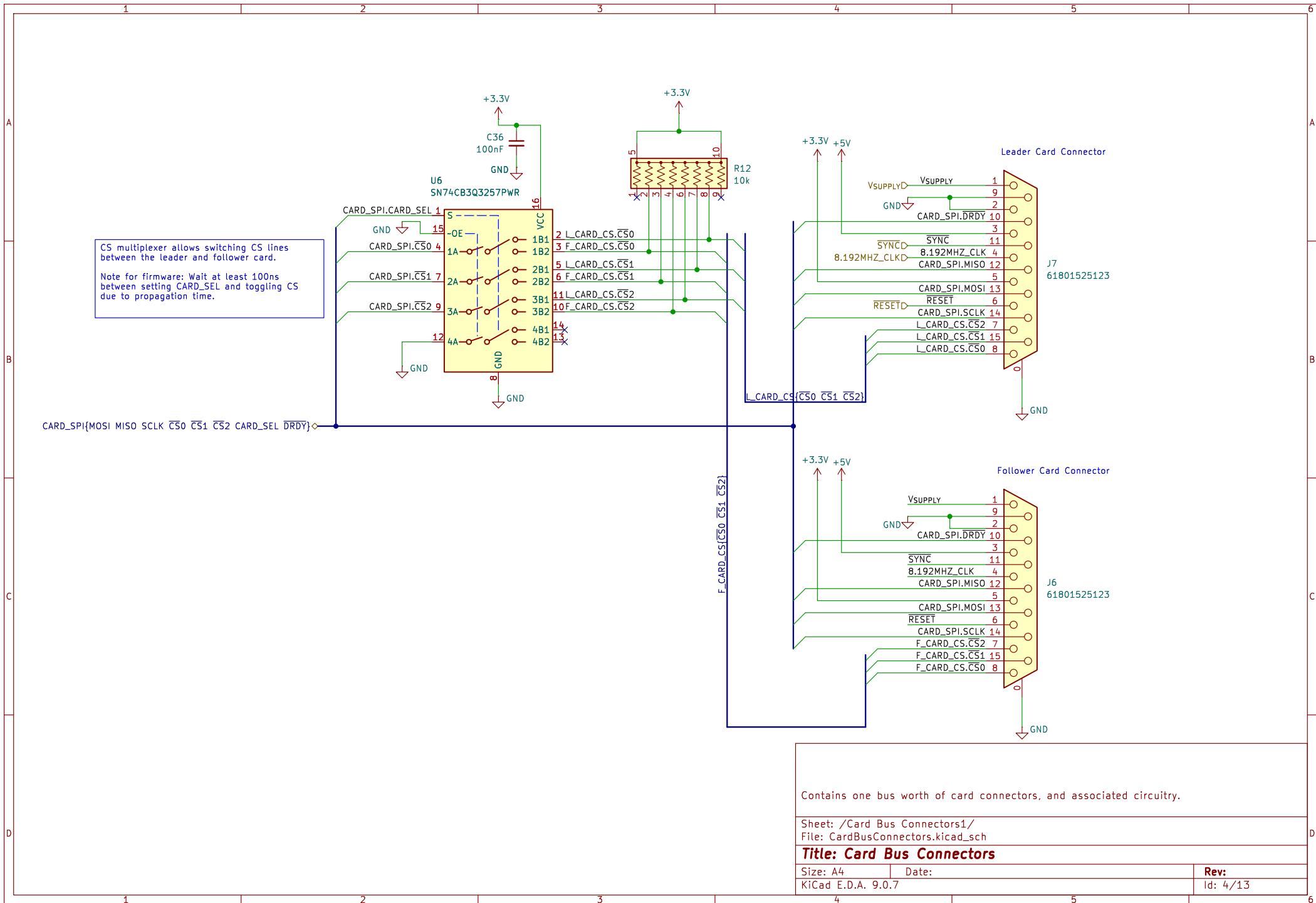
Card Connectors

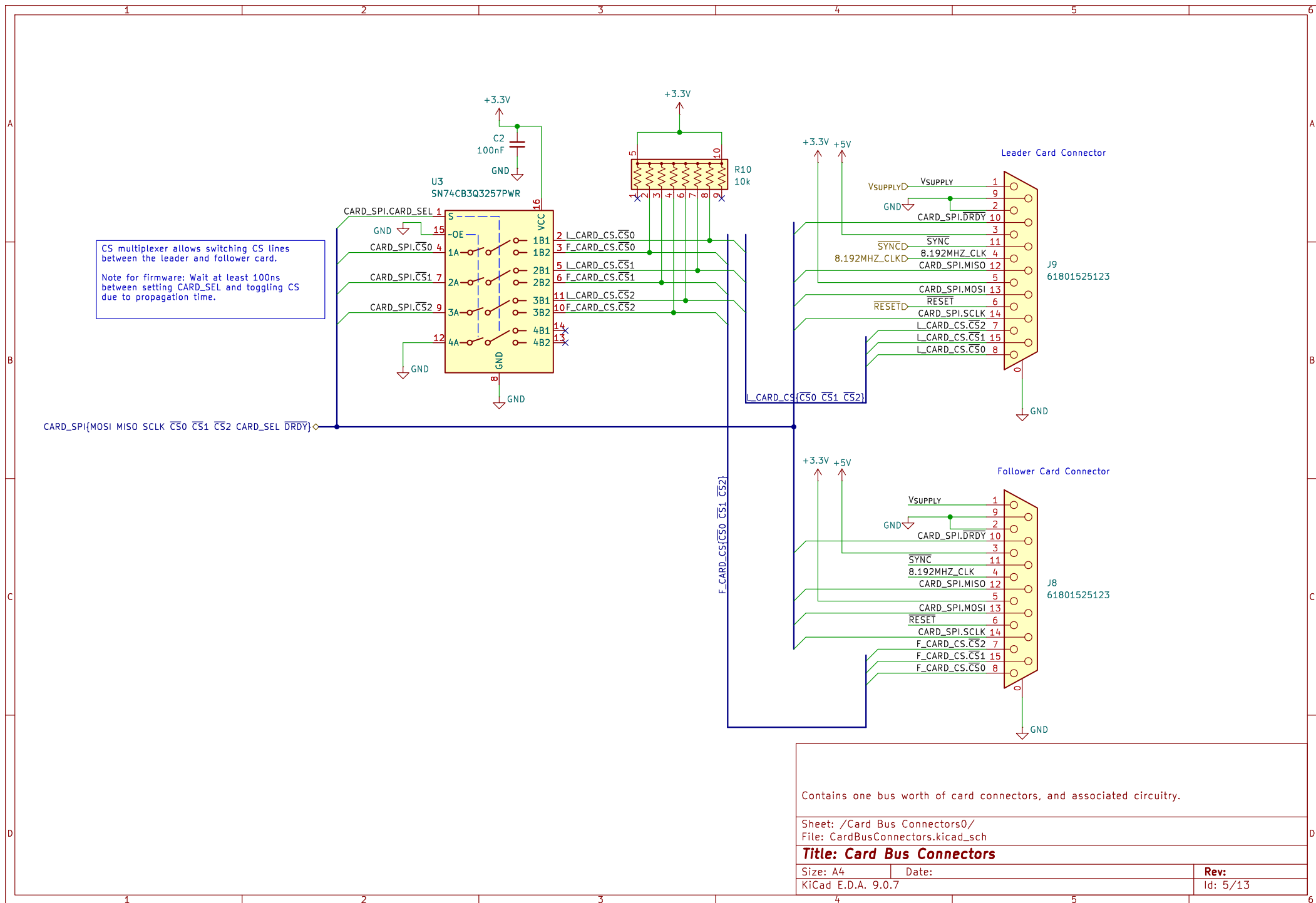


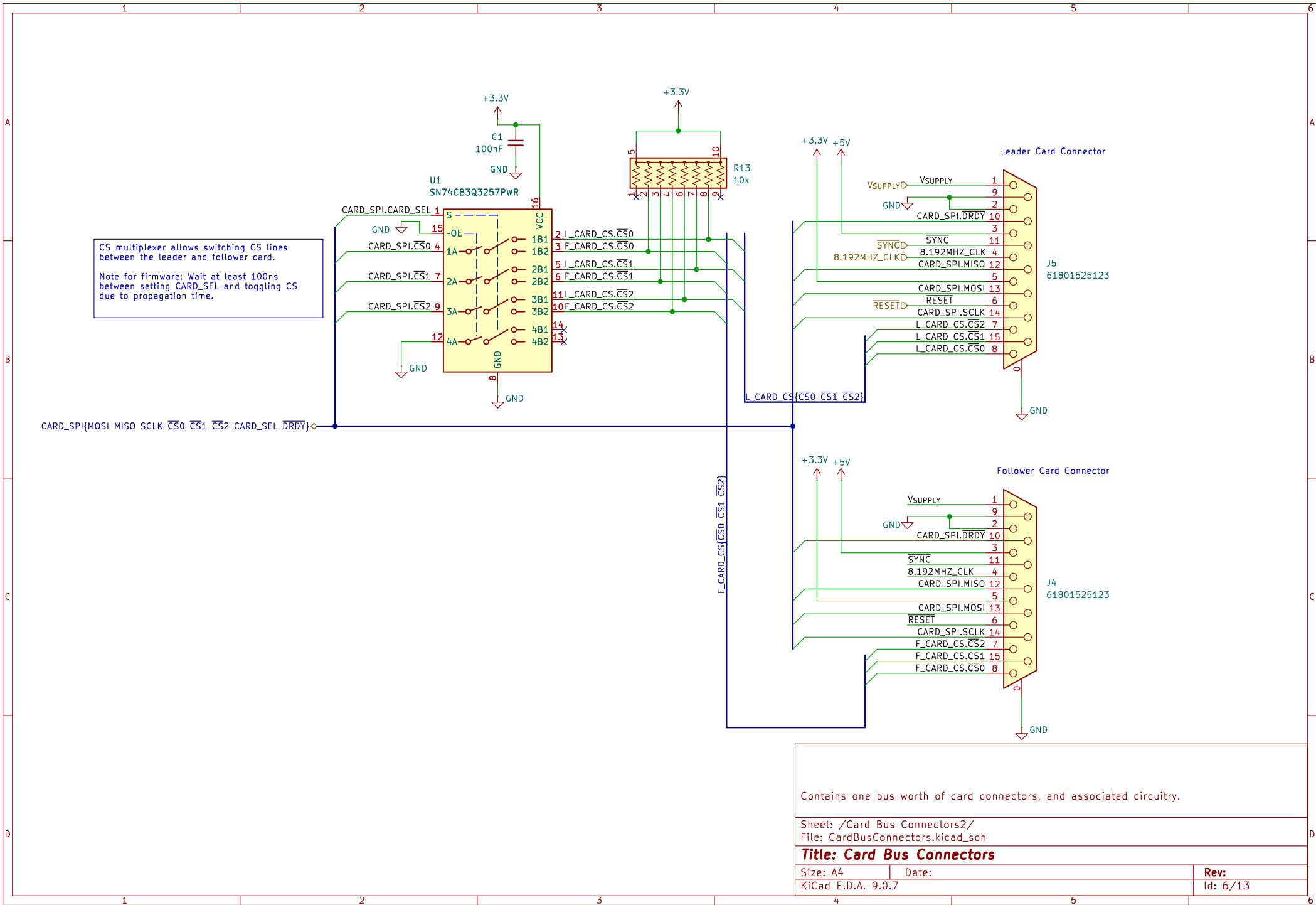
ADC Clock Source











Contains one bus worth of card connectors, and associated circuitry.

Sheet: /Card Bus Connectors2/
File: CardBusConnectors.kicad_sch

Title: Card Bus Connectors

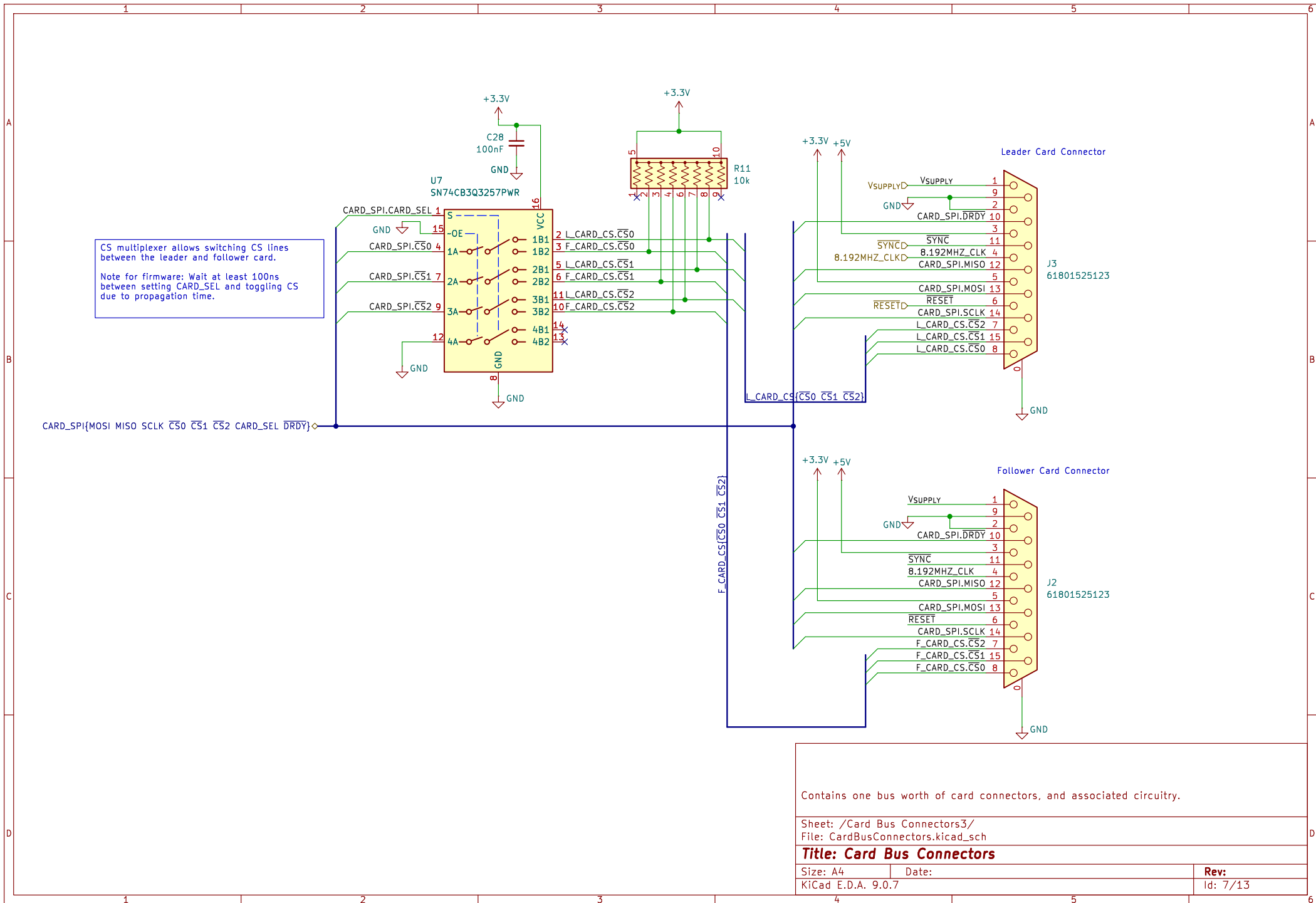
Size: A4

Date:

Rev:

KiCad E.D.A. 9.0.7

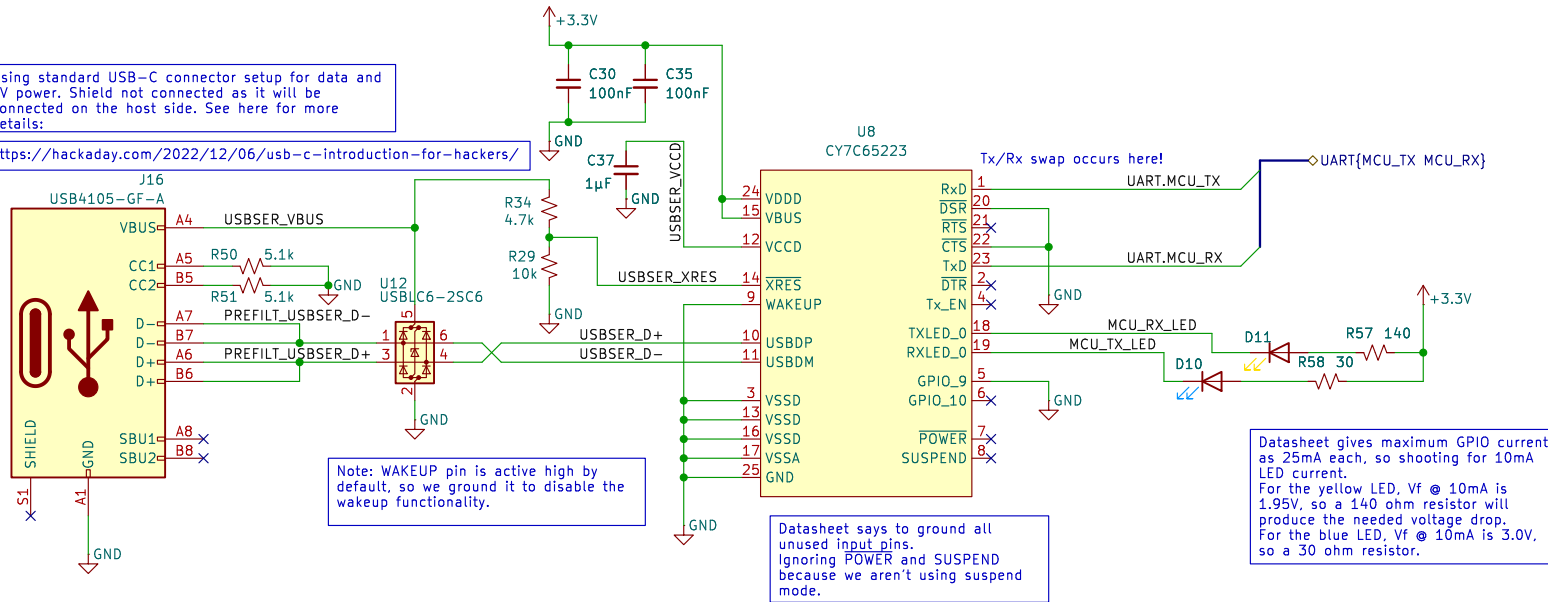
Id: 6/13



This sheet implements a USB-C to UART converter using the Infineon (Cypress) CY7C65223 converter IC.

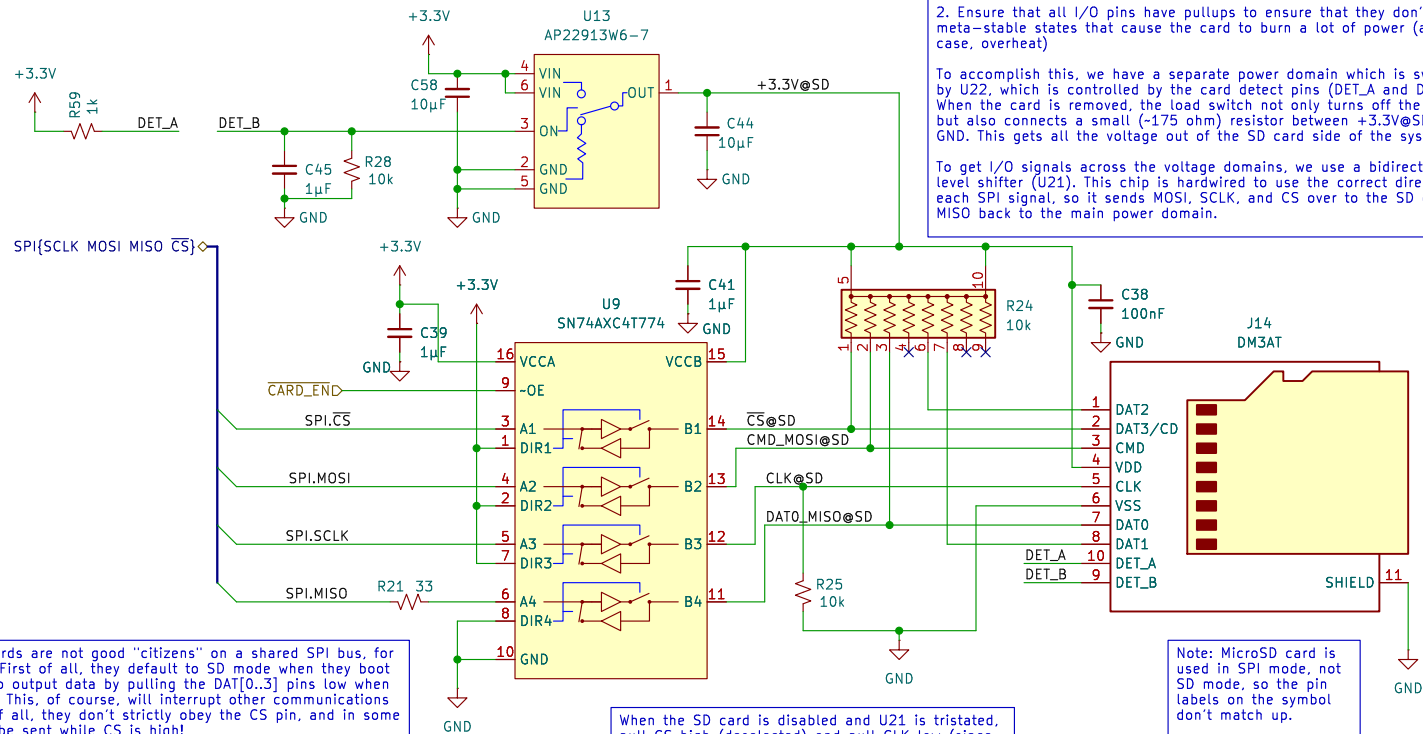
This chip is a favorite of mine because it is very simple to implement in a design: it has an internal EEPROM for storing settings, and does clock recovery from the USB signal so it doesn't need an external crystal. It can also be flashed over USB with a custom serial number/name/description!

<https://hackaday.com/2022/12/06/usb-c-introduction-for-hackers/>



Title: DebugUSBUART

Rev:
Id: 8/13



My friend Kate H gave some advice about how to best protect MicroSD cards. Basically it boils down to:

1. Ensure that there is no power on any of the pins if the card is being removed. This can easily kill SD cards, especially if GND disconnects first so some I/O pins are at a negative voltage relative to the SD card.
2. Ensure that all I/O pins have pullups to ensure that they don't enter meta-stable states that cause the card to burn a lot of power (and, worst case, overheat)

To accomplish this, we have a separate power domain which is switched by U22, which is controlled by the card detect pins (DET_A and DET_B). When the card is removed, the load switch not only turns off the power, but also connects a small (~175 ohm) resistor between +3.3V@SD and GND. This gets all the voltage out of the SD card side of the system ASAP.

To get I/O signals across the voltage domains, we use a bidirectional level shifter (U21). This chip is hardwired to use the correct directions for each SPI signal, so it sends MOSI, SCLK, and CS over to the SD card and MISO back to the main power domain.

Annoyingly, MicroSD cards are not good "citizens" on a shared SPI bus, for a couple of reasons. First of all, they default to SD mode when they boot up, and they can try to output data by pulling the DAT[0..3] pins low when they see clock edges. This, of course, will interrupt other communications on the bus. Second of all, they don't strictly obey the CS pin, and in some cases require data to be sent while CS is high!

So, to fix this, a separate line is used to connect or disconnect the card from the bus entirely by tristating the level shifter. This makes sure it's disconnected from the bus and can't cause trouble when we aren't actively using it.

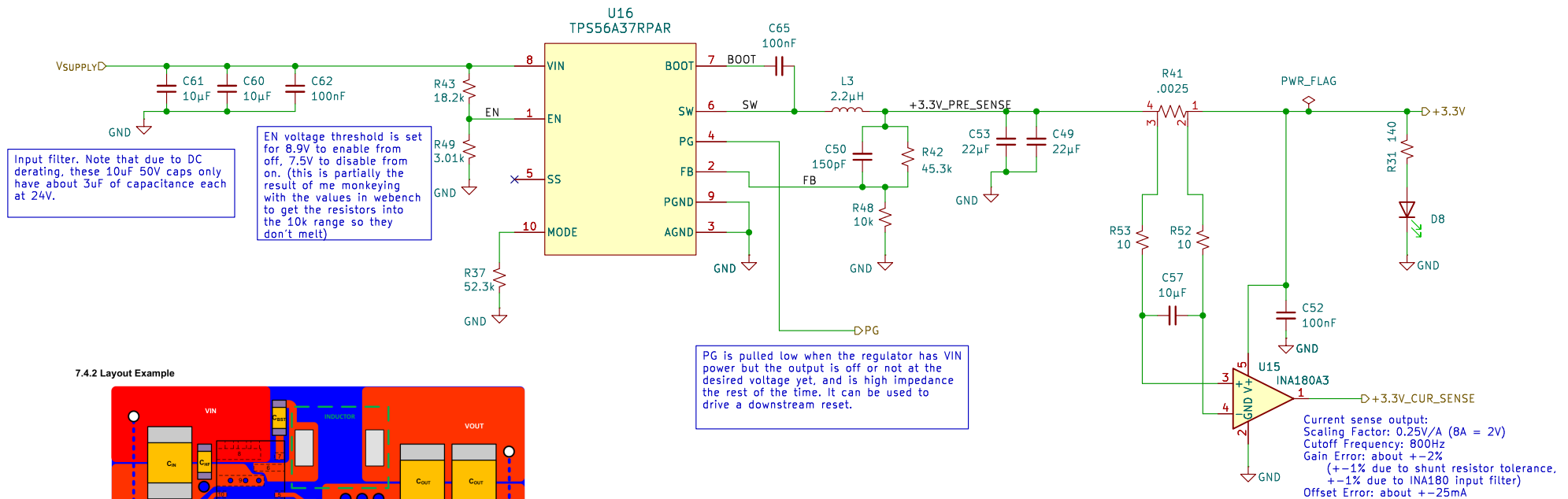
When the SD card is disabled and U21 is tristated, pull CS high (deselected) and pull CLK low (since that's the idle value). This ensures that the SD card won't get any noise that looks like SPI or SD data.

Additionally, pull all of the DAT pins high because the SD card expects this and not doing it can lead to high current consumption. See <https://electronics.stackexchange.com/a/39578>

Note: MicroSD card is used in SPI mode, not SD mode, so the pin labels on the symbol don't match up.

9–24V to 3.3V 8A Regulator

Designed using TI Webench:
<https://webench.ti.com/appinfo/webench/scripts/SDP.cgi?ID=C666EE00B2365BB4>



7.4.2 Layout Example

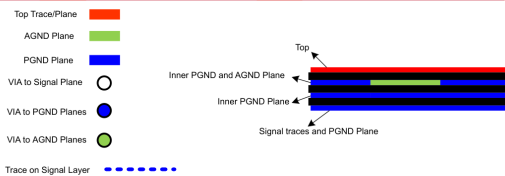
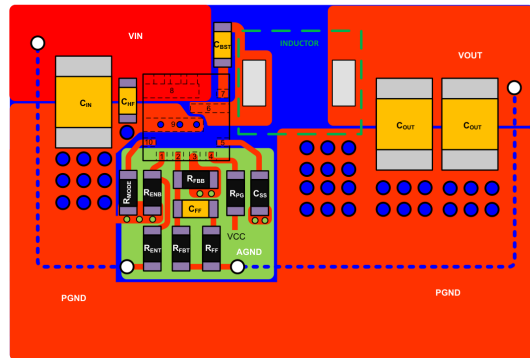


Figure 7-18. TPS56A37 layout

Sheet: /SupplyTo3.3VRegulator/
File: SupplyTo3.3VRegulator.kicad_sch

Title:

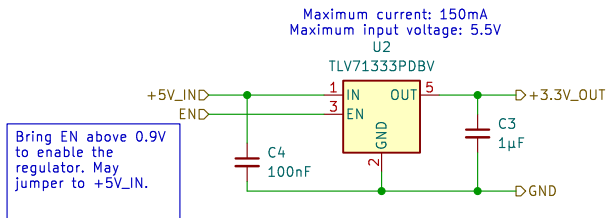
Size: A4

Date:

Rev:

KiCad E.D.A. 9.0.7

Id: 10/13



Bring EN above 0.9V to enable the regulator. May jumper to +5V_IN.

Datasheet recommends using at least 0.1μF input and output capacitors. Figure 9 seems to recommend a 1μF output cap for best PSRR performance.

Sheet: /5V To 3.3V LDO/
File: 5V_To_3.3V_LDO.kicad_sch

Title: 5V To 3.3V LDO

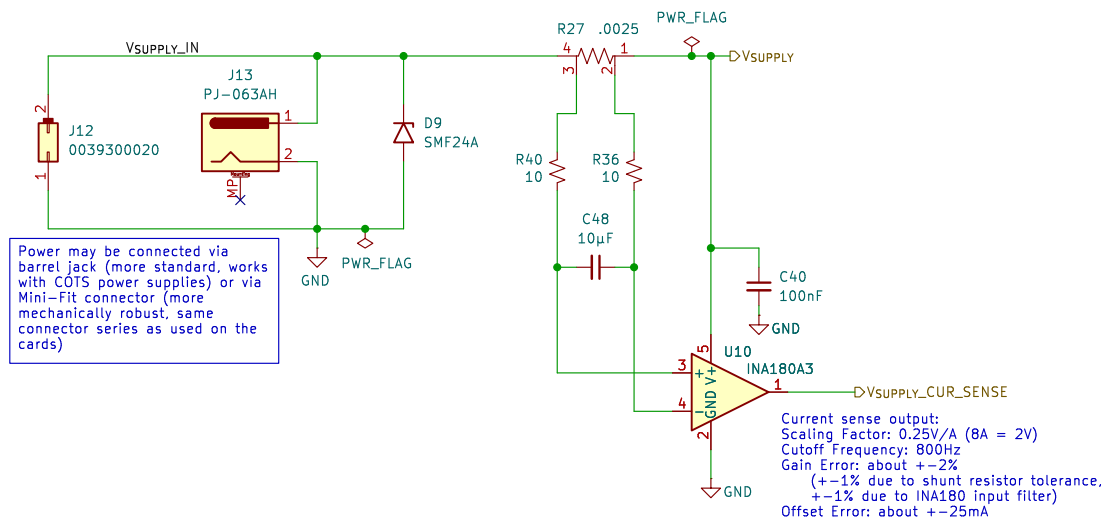
Size: A5
KiCad E.D.A. 9.0.7

Date:

Rev:
Id: 11/13

Approx power budget:
 Assume the following usages:
 - 3.3V regulator: 8A @ 3.3V at 90% efficiency: 29.33W
 - 5V regulator: 8A @ 5V at 90% efficiency: 44.44W
 - Each card: 100mA on VSUPPLY

This gives that Vsupply must be able to supply about 74W + 800mA, or about 4A @VSUPPLY=24V or about 9A @ VSUPPLY=9V. I'm going to fudge that a little and say that our max current is 8A (so that we can use the same current sense circuit as the other rails and so we stay under the limit for this barrel jack).



Sheet: /VSUPPLY Input/
 File: VsupplyInput.kicad_sch

Title:

Size: A4

Date:

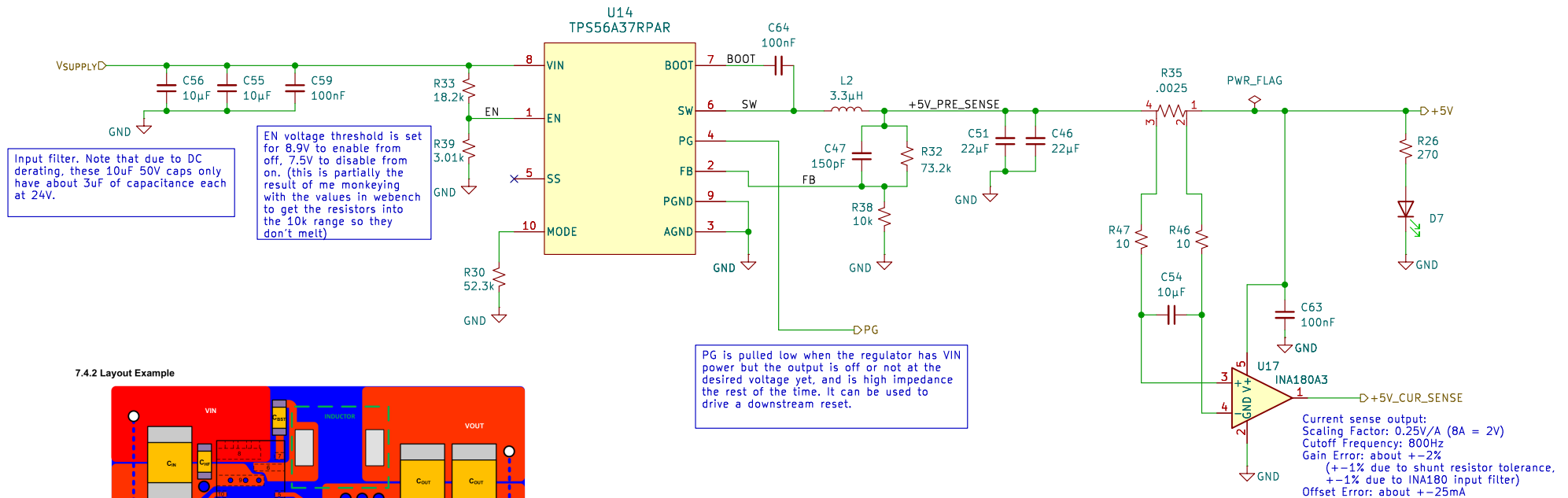
KiCad E.D.A. 9.0.7

Rev:

Id: 12/13

9–24V to 5V 8A Regulator

Designed using TI Webench:
<https://webench.ti.com/appinfo/webench/scripts/SDP.cgi?ID=7F157B1220B5F138>



7.4.2 Layout Example

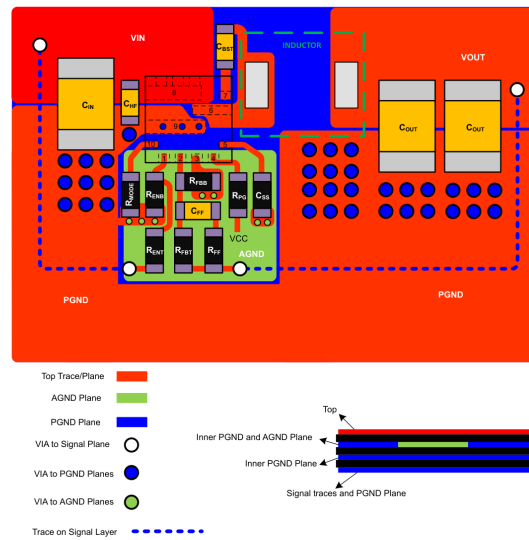


Figure 7-18. TPS56A37 layout

Sheet: /SupplyTo5VRegulator/
 File: SupplyTo5VRegulator.kicad_sch

Title:

Size: A4

Date:

Rev:

KiCad E.D.A. 9.0.7

Id: 13/13